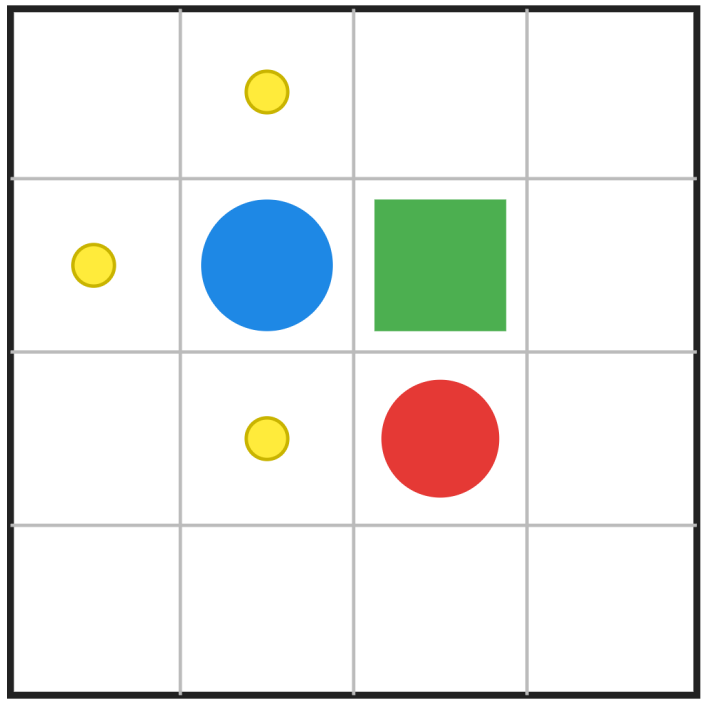
Image AutoGUI Guide

GamesCrafters · GamesmanUni

From a game's rules to a playable interface: design, strings, JSON.



What GamesmanUni needs to draw a game

1. **Image AutoGUI data** (one per variant): the coordinate space, centers, which image each character draws, sounds and animation. Lives in GamesCraftersUWAPI.
2. **A position string** for every position: whose turn it is, plus one character per center.
3. **A move button string** for every legal move: which button to draw, where, and what sound it plays.

Buttons are colored by move value and remoteness automatically. The AutoGUI never needs to know the rules, you just describe where things are.

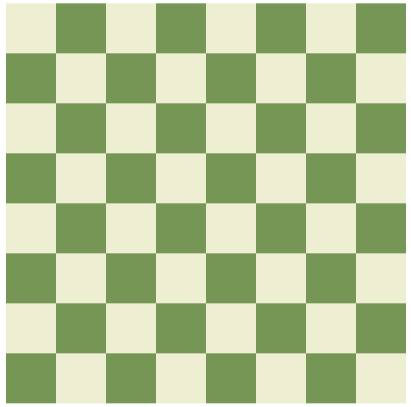
Repos you'll touch

Repo	What you change	How it lands
GamesmanUni	SVGs under src/models/images/svg/<game>/	Branch + pull request
GamesCraftersUWAPI	games/__init__.py and games/image_autogui_games.py	Push to master (org members)
GamesmanClassic (C)	src/m<game>.c: the AutoGUI string functions	Push to master (org members)
GamesmanPuzzles	toString / moveString with StringMode.AUTOGUI	Push or PR
Python game in UWAPI	autoguiPosition / autoguiMove in position_data	Push to master
GamesmanOne	[To be written]	[To be written]

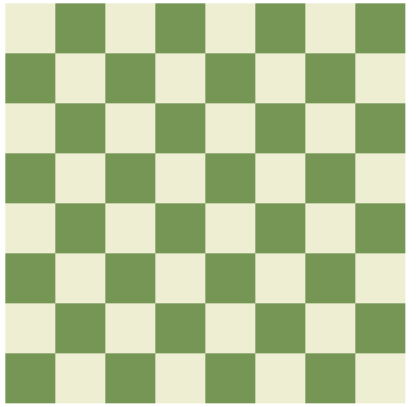
You always touch GamesmanUni and UWAPI, plus **one** of the others depending on where your game is written.

Step 1: Define a Coordinate Space

- Set it with **space: [w, h]**. The numbers are unitless, not pixels. GamesmanUni scales everything to fit the screen.
- (0, 0) is the top-left, (w, 0) the top-right, (0, h) the bottom-left. y goes down.
- Only the aspect ratio matters, and it has to match your background SVG.
- Pick numbers that make centers easy: one unit per cell, or your SVG's viewBox.



space: [8, 8]



space: [62.6, 62.6]

Both look identical on screen. a8's center is [0.5, 0.5] in the first and about [3.9, 3.9] in the second.

Step 2: Identify Entities

Game	Entity	Char
Chess	White king / black king	K / k
Chess	White rook	R
Shift-Tac-Toe	Red piece / black piece	R / B
Shift-Tac-Toe	Slider	S
3-Spot	Red / white / blue half-piece	R / W / B
3-Spot	Score digits	0 … 9
3-Spot	Move tokens	h / v

- An entity is anything that shows up, moves or changes: pieces, sliders, score digits.
- Give each one a single alphanumeric character (case-sensitive).
- Each character maps to one image in **charImages**.
- Unmapped characters draw nothing, so use **-** for empty.

Step 3: Choose Move Button Types

Button	String	Draws	Size
Default token	A_-_ _<sound>	Circle	circleButtonRadius
Custom token	A_<char>_ _<sound>	That char's SVG	its scale
Arrow	M_<from>_<to>_<sound>	Arrow	arrowWidth
Line	L_<c1>_<c2>_<sound>	Line	lineWidth
Text	T_<text>_ _<sound>	Clickable text	textButtonFontSize
Curves (LX)	LQ_… / LC_… / LA_…	Bézier or arc	see LX docs

- The last field is a sound character from **sounds**. Use **-** for no sound.
- Anything that doesn't match these formats becomes a plain button under the board, labeled with the move name.

Step 4: Count Centers

You need a center for every spot an entity can be, every token, and both ends of every arrow or line.

Game	Centers	Breakdown
Chess	64	One per square; arrows reuse them
3-Spot	25	9 half-pieces + 12 move tokens + 4 score digits
L-game	64	16 squares + 48 move tokens
Shift-Tac-Toe	42	Pieces, sliders and arrow ends (0–41)

Watch out: the position string has one character per center, so if two entities can sit on the same spot at the same time (like a slider under a piece), they each need their own center there.

Step 5: Position Strings

```
# Achi, "basic" theme
"space": [100, 100],
"centers": [
  [10,10], [50,10], [90,10],
  [10,50], [50,50], [90,50],
  [10,90], [50,90], [90,90]],
"charImages": {
  "o": {"image": "general/whitepiece.svg",
        "scale": 15},
  "x": {"image": "general/blackpiece.svg",
        "scale": 15}}
```

Format: **<turn>_<entities>**

2_-x-o-ox-o

- Turn is 1 or 2.
- Character i is drawn at centers[i]. Index 1 is "x" and centers[1] is [50, 10], so blackpiece.svg goes there.
- Characters not in charImages are skipped.

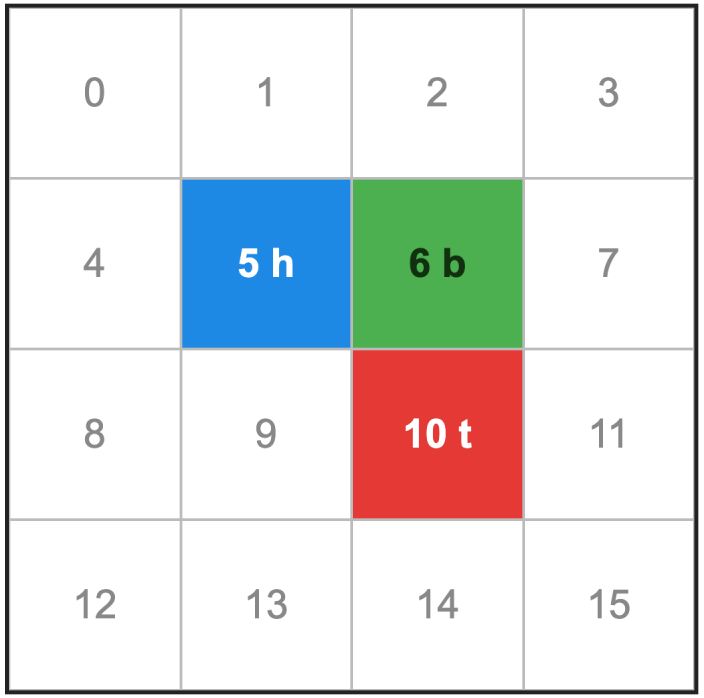
Step 6: Move Button Strings

String	Game	Draws
A_-_7_x	Achi	Default circle at center 7; clicking plays sound x
A_h_3_-	3-Spot	The SVG mapped to "h" at center 3, silent
M_0_4_x	Nu Tic-Tac-Toe	Arrow from center 0 to center 4
L_61_77_x	Quick Cross	Line between 61 and 77 (order doesn't matter)
T_A_1_x	Ghost	Clickable "A" at center 1

Example: 4x4 Snake

Player 1 moves the **head**, player 2 moves the **tail**, one square at a time (no diagonals) onto an empty square. The square left behind becomes **body**. You lose if you can't move.

Decision	Choice
Entities	h head, b body, t tail
space	[4, 4], one unit per cell
centers	16, row by row
Move buttons	Arrows (or default tokens)



Reference: Theme Fields

Field	What it sets
space	[w, h] coordinate space
centers	List of [x, y] points
charImages	char → {image, scale}
background	Image behind everything
foreground	Image over entities
sounds	char → sound file
animationType	entityFade or simpleSlides
defaultAnimationWindow	[start, end) to animate

Optional	Default
circleButtonRadius	0.02 × space width
arrowWidth	0.015 × space width
lineWidth	0.04 × space width
textEntityFontSize	0.1 × space width
textButtonFontSize	0.1 × space width
entitiesOverArrows	false
ambience *	none (looping sound)
defaultTheme *	required

* Top level, next to "themes". Image paths are relative to GamesmanUni/src/models/images/svg/, ambience to src/models/sounds/.

Reference: Sizes and Units

- Every size is in the same units as **space**.
- Images: each SVG is resized so its larger side equals **scale**.
- Buttons: circleButtonRadius, arrowWidth and lineWidth work the same way.
- Text: font size s renders at 100 · s / w px.

Achi, space [100, 100]

Field	Value
centers	10, 50, 90
piece scale	15
circleButtonRadius	6.5
arrowWidth	4

Pieces are 15% of the board width. On Snake's [4, 4], scale 1 is exactly one cell.

Reference: Drawing Order

top

Move buttons
Foreground image
Entities (and animations)
Background image

bottom

- A foreground lets pieces slide behind part of the board. Shift-Tac-Toe's frame is a foreground with holes in it.
- **entitiesOverArrows: true** draws pieces on top of arrows (Achi does this).
- Background and foreground need the same aspect ratio as space.

Tips

- Reuse what's there: svg/general/ has plain pieces, svg/chess/ has a full set.
- Keep SVGs small and hand-written. Exports from design tools carry a lot of extra code.
- Set space to your background's viewBox so you can read centers straight off the drawing.
- Touching tiles show thin gaps at scale 1. Use 1.01 (Chomp does this).
- Half of a stroke falls outside its shape, so inset it or widen the viewBox.
- Get the strings right in --interact before debugging anything in the browser.